

## 3651. Minimum Cost Path with Teleportations

Solved 

Hard

 Topics

 Companies

 Hint

You are given a  $m \times n$  2D integer array `grid` and an integer `k`. You start at the top-left cell  $(0, 0)$  and your goal is to reach the bottom-right cell  $(m - 1, n - 1)$ .

There are two types of moves available:

- **Normal move:** You can move right or down from your current cell  $(i, j)$ , i.e. you can move to  $(i, j + 1)$  (right) or  $(i + 1, j)$  (down). The cost is the value of the destination cell.
- **Teleportation:** You can teleport from any cell  $(i, j)$ , to any cell  $(x, y)$  such that `grid[x][y] <= grid[i][j]`; the cost of this move is 0. You may teleport at most `k` times.

Return the **minimum** total cost to reach cell  $(m - 1, n - 1)$  from  $(0, 0)$ .

### Example 1:

**Input:** `grid = [[1,3,3],[2,5,4],[4,3,5]]`, `k = 2`

**Output:** 7

**Explanation:**

Initially we are at  $(0, 0)$  and cost is 0.

| Current Position | Move                 | New Position | Total Cost  |
|------------------|----------------------|--------------|-------------|
| $(0, 0)$         | Move Down            | $(1, 0)$     | $0 + 2 = 2$ |
| $(1, 0)$         | Move Right           | $(1, 1)$     | $2 + 5 = 7$ |
| $(1, 1)$         | Teleport to $(2, 2)$ | $(2, 2)$     | $7 + 0 = 7$ |

The minimum cost to reach bottom-right cell is 7.

### Example 2:

**Input:** `grid = [[1,2],[2,3],[3,4]], k = 1`

**Output:** 9

#### Explanation:

Initially we are at (0, 0) and cost is 0.

| Current Position | Move       | New Position | Total Cost  |
|------------------|------------|--------------|-------------|
| (0, 0)           | Move Down  | (1, 0)       | $0 + 2 = 2$ |
| (1, 0)           | Move Right | (1, 1)       | $2 + 3 = 5$ |
| (1, 1)           | Move Down  | (2, 1)       | $5 + 4 = 9$ |

The minimum cost to reach bottom-right cell is 9.

### Constraints:

- $2 \leq m, n \leq 80$
- $m == \text{grid.length}$
- $n == \text{grid}[i].\text{length}$
- $0 \leq \text{grid}[i][j] \leq 10^4$
- $0 \leq k \leq 10$

## Python:

class Solution:

```
def minCost(self, grid: List[List[int]], k: int) -> int:
    # dp[i][j][n_teleport]: list[list[int]] =>
    # minimum cost to get to (i, j) by using at most n_teleport teleportations
    # Size of dp => m*n*k
```

```

# Initialization:
# dp[i][j][0] = min(
#     dp[i-1][j][0], # move down
#     dp[i][j-1][0] # move right
# ) + grid[i][j]

# Transitions:
# dp[i][j][n_teleport] = min(
#     dp[i-1][j][n_teleport] + grid[i][j], # move down
#     dp[i][j-1][n_teleport] + grid[i][j], # move right
#     min(
#         dp[ip][jp][n_teleport-1]
#     ) for all ip, jp s.t. grid[ip][jp] >= grid[i][j]
# )
# To compute:
#     min(
#         dp[ip][jp][n_teleport-1]
#     ) for all ip, jp s.t. grid[ip][jp] >= grid[i][j]
# We have to be clever otherwise TLE

# We will use min_cost_from_cells_of_value_v: list[int] of size max(grid) + 1
# min_cost_from_cells_of_value_v[v] = minimum cost from all the dp[i][j][n_teleport - 1] s.t.
grid[i][j] = v
# This can be computed in O(max(grid) + m*n)

# We will use min_cost_from_cells_above_value_v: list[int] of size max(grid) + 1
# min_cost_from_cells_above_value_v[v] = minimum cost from all the dp[i][j][n_teleport - 1]
s.t. grid[i][j] >= v
# This can be computed in O(max(grid))
# min_cost_from_cells_above_value_v[v] = min(
#     min_cost_from_cells_above_value_v[v+1],
#     min_cost_from_cells_of_value_v[v]
# )

dp: list[list[list[int]]] = [
    [
        [
            10**18 for _ in range(k+1)
        ] for _ in range(len(grid[0]))
    ] for _ in range(len(grid))
]

dp[0][0][0] = 0
for i in range(len(grid)):

```

```

for j in range(len(grid[0])):
    if i != 0 or j != 0:
        dp[i][j][0] = min(
            dp[i-1][j][0] if i != 0 else 10**18, # move down
            dp[i][j-1][0] if j != 0 else 10**18 # move right
        ) + grid[i][j]

max_grid: int = 0
for i in range(len(grid)):
    for j in range(len(grid[0])):
        max_grid = max(max_grid, grid[i][j])

for n_teleport in range(1, k+1):
    min_cost_from_cells_of_value_v: list[int] = [10**18 for _ in range(max_grid + 1)]
    for i in range(len(grid)):
        for j in range(len(grid[0])):
            min_cost_from_cells_of_value_v[grid[i][j]] = min(
                min_cost_from_cells_of_value_v[grid[i][j]],
                dp[i][j][n_teleport-1]
            )
    min_cost_from_cells_above_value_v: list[int] = [10**18 for _ in range(max_grid + 1)]
    min_cost_from_cells_above_value_v[max_grid] =
min_cost_from_cells_of_value_v[max_grid]
    for v in range(max_grid-1, -1, -1):
        min_cost_from_cells_above_value_v[v] = min(
            min_cost_from_cells_above_value_v[v+1],
            min_cost_from_cells_of_value_v[v]
        )

    for i in range(len(grid)):
        for j in range(len(grid[0])):
            dp[i][j][n_teleport] = min(
                (dp[i-1][j][n_teleport] + grid[i][j]) if i != 0 else 10**18, # move down
                (dp[i][j-1][n_teleport] + grid[i][j]) if j != 0 else 10**18, # move right
                min_cost_from_cells_above_value_v[grid[i][j]]
            )

return dp[-1][-1][k]

```

## JavaScript:

```

/**
 * @param {number[][]} grid
 * @param {number} k
 * @return {number}

```

```

*/
var minCost = function(grid, k) {
  const R = grid.length, C = grid[0].length, RC = R * C,
    RMinus1 = R - 1, CMinus1 = C - 1, RCMinus1 = RC - 1,
    kPlus1 = 1 + k
  const memo = new Array(R)
  for (const i of memo.keys()) {
    const row = memo[i] = new Array(C)
    for (const j of row.keys())
      row[j] = []
  }

  // key: prepare for directed DP
  const MinFL = new Array(kPlus1)
  for (const rmg of MinFL.keys()) {
    const row = MinFL[rmg] = new Array(RC)
    row[-1] = Infinity
  }

  const items = []
  for (const [i, row] of grid.entries()) {
    for (const [j, val] of row.entries()) {
      const item = [i, j, val]
      items.push(item)
    }
  }
  items.sort((A, B) => A[2] - B[2])

  let costSF = Infinity
  const cost2lastIndex = new Array(RC)
  for (let i = RCMinus1; i > -1; i--) {
    const cost = items[i][2]
    if (cost < costSF) {
      costSF = cost
      cost2lastIndex[cost] = i
    }
  }

  function dp(fromI, fromJ, rmg) {
    if (fromI === RMinus1 && fromJ === CMinus1)
      return 0

    const existing = memo[fromI][fromJ][rmg]

```

```

    if (existing !== undefined)
        return existing

    let result = Infinity
    const val = grid[fromI][fromJ]
    if (fromI < RMinus1) {
        const outcomeR = grid[fromI + 1][fromJ] + dp(fromI + 1, fromJ, rmg)
        result = Math.min(result, outcomeR)
    }
    if (fromJ < CMinus1) {
        const outcomeC = grid[fromI][fromJ + 1] + dp(fromI, fromJ + 1, rmg)
        result = Math.min(result, outcomeC)
    }

    if (rmg) {
        const rmgMinus1 = rmg - 1
        // key: directed DP
        const minFL = MinFL[rmgMinus1]
        const reachment = cost2lastIndex[val]

        const subresult = minFL[reachment]
        result = Math.min(result, subresult)
    }

    return memo[fromI][fromJ][rmg] = result
}
for (let rmg = 0; rmg < kPlus1; rmg++) {
    const minFL = MinFL[rmg]
    // key: directed DP
    // the order is critical!
    for (const [sf, item] of items.entries()) {
        const [i, j] = item
        const outcome = dp(i, j, rmg)
        minFL[sf] = Math.min(minFL[sf - 1], outcome)
    }
}

const result = dp(0, 0, k)
return result
};

```

**Java:**

```

class Solution {

```

```

public int minCost(int[][] grid, int k) {
    int n = grid.length, m = grid[0].length, max = 0;
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < m; j++) max = Math.max(max, grid[i][j]); //find max val in grid
    }

    int[][] dp = new int[n][m]; //dp[i][j] = min cost for cell (i, j) to reach cell (m - 1, n - 1)
    int[] temp = new int[max + 1], best = new int[max + 1]; //temp[val] = min cost for a cell with
a value of val to reach cell (m - 1, n - 1)
    Arrays.fill(temp, Integer.MAX_VALUE);
    temp[grid[n - 1][m - 1]] = 0;

    for(int i = m - 2; i >= 0; i--) { //handle the bottom row seperately
        dp[n - 1][i] = dp[n - 1][i + 1] + grid[n - 1][i + 1];
        temp[grid[n - 1][i]] = Math.min(temp[grid[n - 1][i]], dp[n - 1][i]);
    }
    for(int i = n - 2; i >= 0; i--) {
        dp[i][m - 1] = dp[i + 1][m - 1] + grid[i + 1][m - 1]; //handle the right column seperately
        temp[grid[i][m - 1]] = Math.min(temp[grid[i][m - 1]], dp[i][m - 1]);
        for(int j = m - 2; j >= 0; j--) {
            dp[i][j] = Math.min(dp[i + 1][j] + grid[i + 1][j], dp[i][j + 1] + grid[i][j + 1]); //fill out base
table with no teleports
            temp[grid[i][j]] = Math.min(temp[grid[i][j]], dp[i][j]); //update min cost for each grid value
        }
    }

    for(int x = 0; x < k; x++) { //for each possible teleport, dp[i][j] = min cost to go from cell (i, j)
to cell (m - 1, n - 1) using at most x teleports
        //cost[i][j] = min(cost[i + 1][j], cost[i][j + 1], best[grid[i][j]])

        best[0] = temp[0];
        for(int i = 1; i <= max; i++) best[i] = Math.min(best[i - 1], temp[i]); //best[val] = min cost for
a cell with a value <= val to reach the cell (m - 1, n - 1)

        for(int i = m - 2; i >= 0; i--) { //handle the bottom row seperately
            dp[n - 1][i] = Math.min(best[grid[n - 1][i]], dp[n - 1][i + 1] + grid[n - 1][i + 1]);
            temp[grid[n - 1][i]] = Math.min(temp[grid[n - 1][i]], dp[n - 1][i]);
        }
        for(int i = n - 2; i >= 0; i--) {
            dp[i][m - 1] = Math.min(best[grid[i][m - 1]], dp[i + 1][m - 1] + grid[i + 1][m - 1]); //handle
the right column seperately
            temp[grid[i][m - 1]] = Math.min(temp[grid[i][m - 1]], dp[i][m - 1]);
            for(int j = m - 2; j >= 0; j--) {

```

```

        dp[i][j] = Math.min(Math.min(dp[i + 1][j] + grid[i + 1][j], dp[i][j + 1] + grid[i][j + 1]),
best[grid[i][j]]);
        temp[grid[i][j]] = Math.min(temp[grid[i][j]], dp[i][j]);
    }
}
}
return dp[0][0]; //return final cost to go from cell (0, 0) to cell (m - 1, n - 1) using at most k
teleports
}
}

```